I used GitHub Copilot inside VS Code as an inline assistant. Copilot suggests code as I type, and I accept or reject each suggestion as I see fit. It helped me write this enhancement, but the final output you see is the outcome of my modifications. Because this is a maintenance task, I added the new channel in its own file, whatsapp_service.py, and left the engine (the NotificationMedium interface and the AlertSystem class in notification_system.py) unchanged.

Copilot is driven by the surrounding code rather than a chat box, so the prompt was the new file plus the class declaration I wrote to guide it. In whatsapp_service.py I imported the existing interface and typed:

from notification_system import NotificationMedium class WhatsAppService(NotificationMedium): def send(self, message: str) -> str:

Copilot suggested the body. I inspected and then accepted the return line after fixing it to the exact format the assignment requires: [WhatsApp] Sending message: <message>.

How I kept the signature matched to avoid rewriting

- I imported NotificationMedium from the engine module and subclassed it, so the new channel is bound to the one interface already in the system instead of a second copy.
- I rejected any Copilot suggestion that drifted from the interface, for example, versions that renamed a parameter or added extra arguments. The method had to stay send(self, message: str) -> str to match.
- Because send is marked abstract on the base class, Python will not instantiate a subclass that fails to implement it. I confirmed with isinstance() that WhatsAppService is recognized as a NotificationMedium before relying on it.
- I did not change the AlertSystem class or the NotificationMedium interface. The new channel is selected through the same set_medium() and notify_user() that were already there, and it is wired into the menu in the driver file (notification_app.py), not the engine. Adding a file rather than editing the engine keeps this a maintenance change rather than a rewrite.

Generating the test script I also used Copilot to write the test script, maintenance_demo.py, which proves the new channel works at runtime. I described the AlertSystem API in a comment that it takes a medium, has set_medium(), notify_user(), which returns a confirmation string, and get_log(), which returns a list, and let Copilot draft a main() that creates one AlertSystem, switches it across EmailService, SMSService, and WhatsAppService using set_medium(), and prints the shared log. I kept only the suggestions that went through those existing public

methods, so the test exercises the system the same way a real caller would, without reaching into AlertSystem's internals or modifying it.

Docstring adjustments

- I rewrote Copilot's comment into the same docstring style that I used for the rest of the project. A module header, a class docstring, and a method docstring with Parameters and Returns sections.
- I removed the suggestion's redundant inline comment that just repeated the docstring, and kept the return type annotation consistent with EmailService and SMSService.
- I noted in the module docstring that this is a Week 5 maintenance addition that imports and implements the existing interface.